

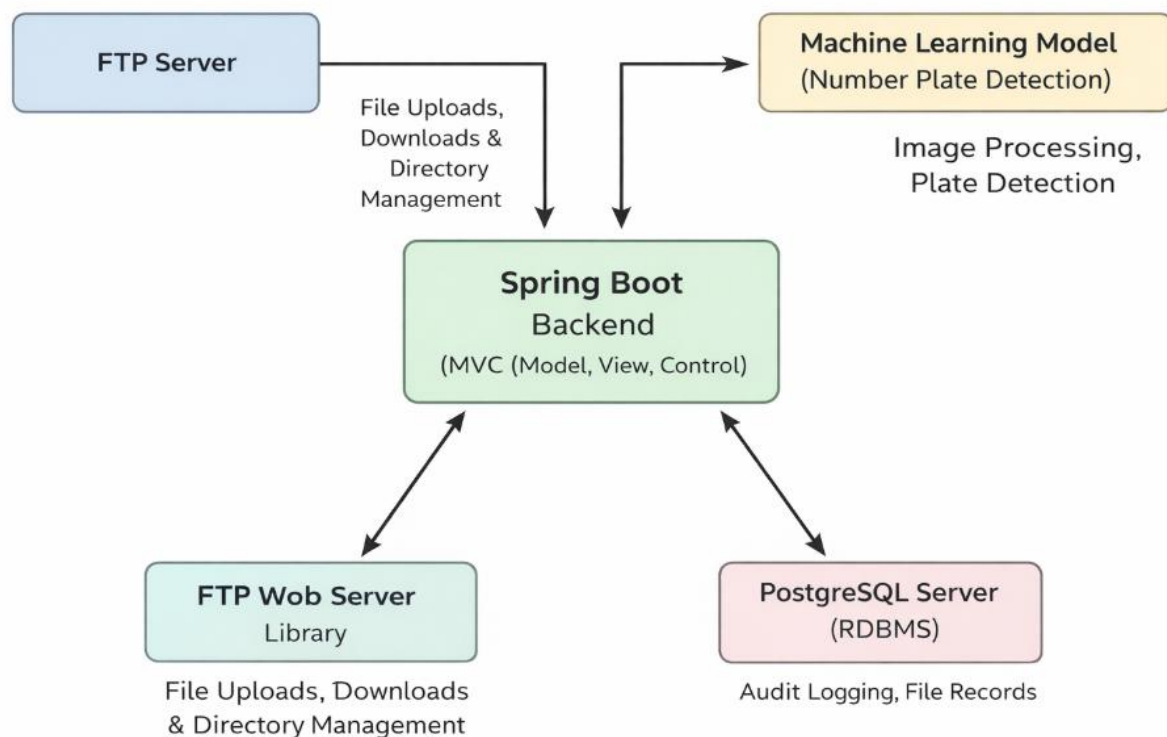
Name: Kishan Patel

Enroll: 25MML0030

Numberplate Detection: Back-end

Abstract

This project focuses on developing a backend system for number plate detection along with efficient file management. The backend is built using the Spring Boot framework and provides REST APIs that allow users to upload vehicle images, retrieve stored images, and download folders from a remote FTP server. When an image is uploaded, it is automatically processed by a machine learning model that detects the vehicle's number plate and generates an output image with highlighted bounding boxes and labels. The system also maintains a database record of every uploaded file using PostgreSQL and Hibernate, storing details such as the file name, storage path, and timestamp. This helps ensure proper tracking of system activities and improves reliability. Overall, the backend acts as a bridge between the client application, the detection model, and the FTP storage system, enabling organized storage, automated processing, and secure data management.



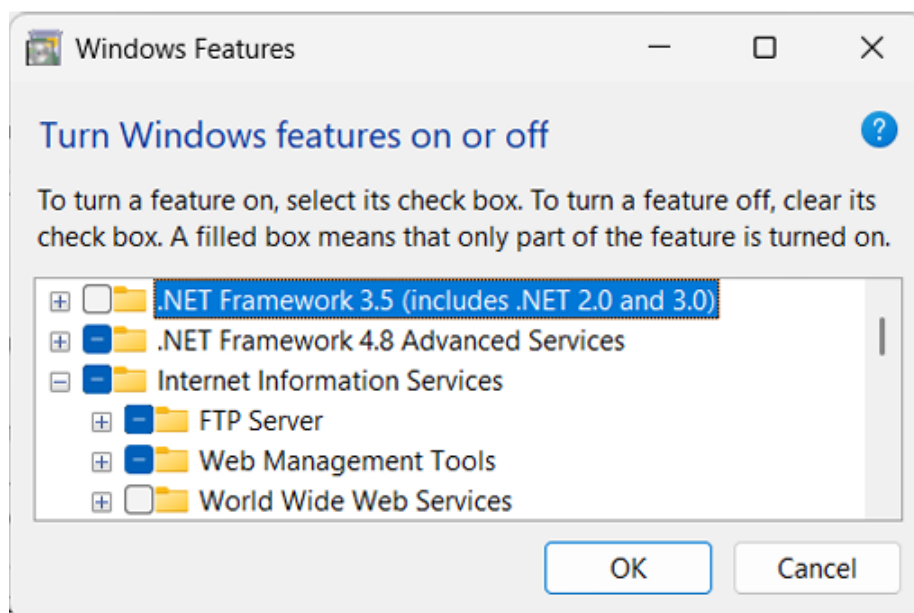
1. System Architecture and Technologies

The backend system is designed using a **layered architecture** that separates different components of the application such as controllers, business logic, and data management. This separation improves code organization, makes the system easier to maintain, and allows each layer to handle its specific responsibilities efficiently.

- **Backend Framework:** The backend is developed using **Spring Boot (Java)**, which provides a robust framework for building scalable web applications and REST APIs. It also simplifies dependency management and configuration through its built-in dependency injection mechanism.
- **Database and ORM:** The system uses **PostgreSQL** as the relational database to store application data. Database operations are managed using **Spring Data JPA along with Hibernate**, which acts as the Object-Relational Mapping (ORM) tool. Hibernate's automatic schema update feature (ddl-auto=update) is used to create and update database tables during application startup.
- **FTP Integration:** Communication with the remote FTP server is handled using the **Apache Commons Net library**. The FTPClient class from this library enables the backend to connect to the server, navigate directories, and perform file operations such as uploading and downloading images.

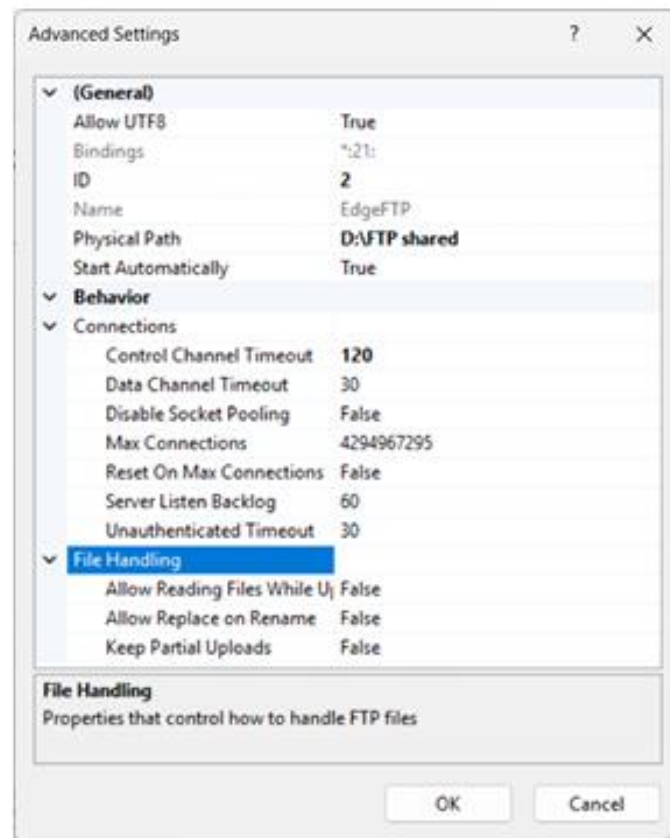
2. Windows Server Infrastructure Setup

To enable file transfer functionality for the backend system, a **local FTP environment** was configured using **Internet Information Services (IIS)** on a Windows operating system. Initially, the required FTP components were enabled through **Windows Features**, including the **FTP Server and Web Management Tools**. This step allows the system to host and manage FTP services directly from the Windows machine.



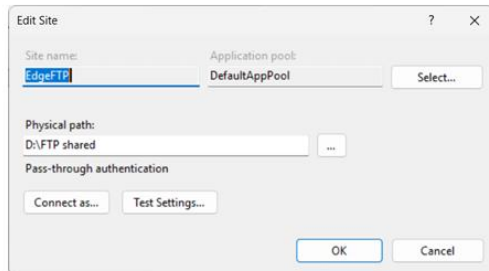
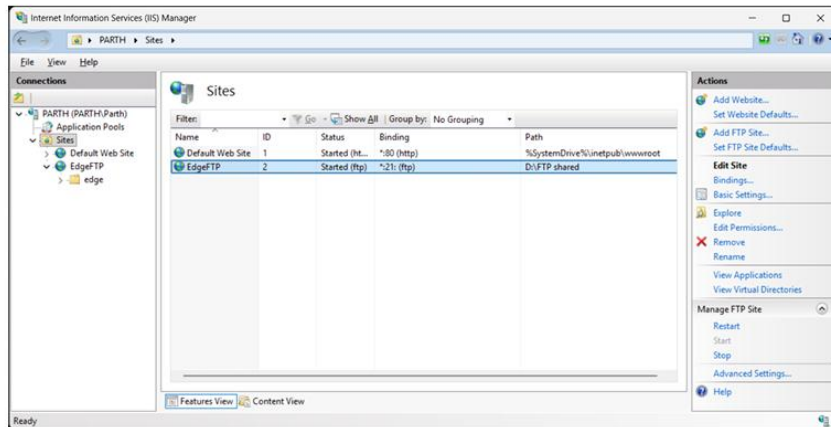
After enabling the required features, a new **FTP site** was created using the **IIS Manager**. The site was configured with a specific name and linked to a designated **physical directory path (D:\FTP shared)** where the uploaded files are stored. The FTP site was bound to the default FTP port **21**, allowing the backend service to communicate with the FTP server.

To ensure secure access to the FTP server, **Basic Authentication** was enabled. Since IIS FTP relies on **Windows system accounts**, a dedicated local user account was created through the Windows command line to handle FTP operations. This account was then assigned the necessary permissions to access and manage files within the FTP directory.

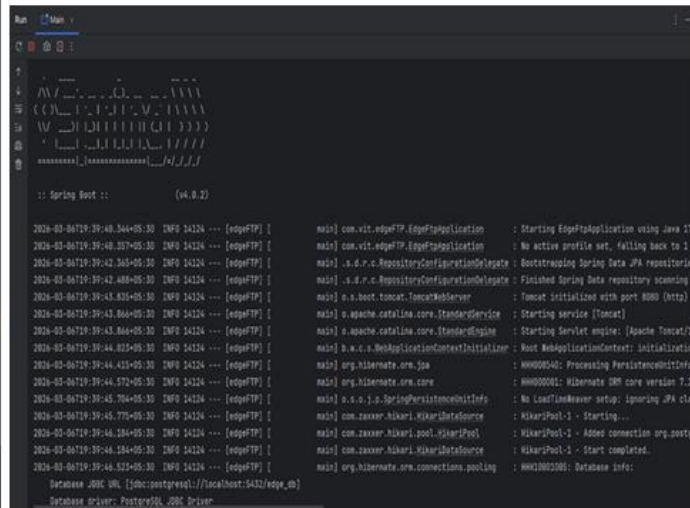
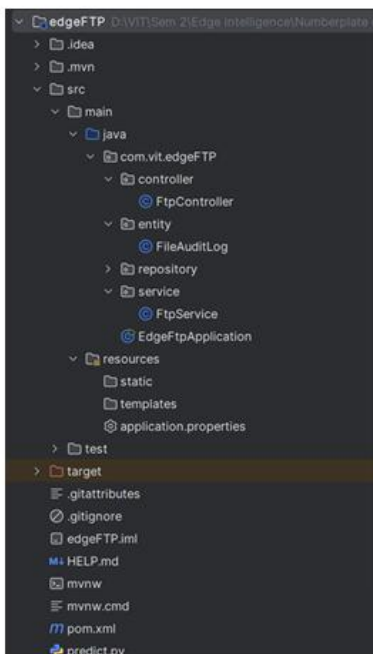


Additionally, **FTP Authorization Rules** were configured to grant the created user **Read and Write permissions**. The NTFS file system permissions of the storage folder were also modified to provide the user with **Full Control**, ensuring that the backend application can successfully upload, retrieve, and manage files.

With these configurations in place, the **Spring Boot backend application** can connect to the FTP server using the FTP client library. This setup allows the system to automatically create directories, upload images, and manage file storage without encountering authentication or access issues.



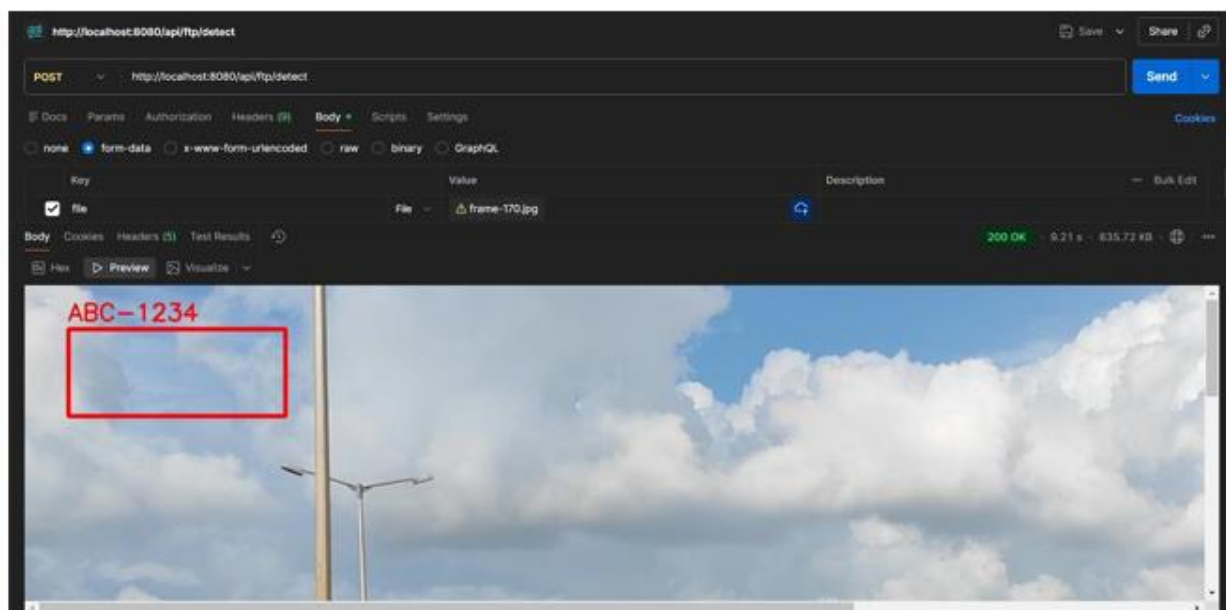
3. Core API Functionalities



The backend service provides several **RESTful API endpoints** that handle file management and image processing tasks. These APIs enable communication between the client application, the FTP storage system, and the machine learning model. Custom HTTP headers are used to dynamically specify file locations and manage operations on the remote server.

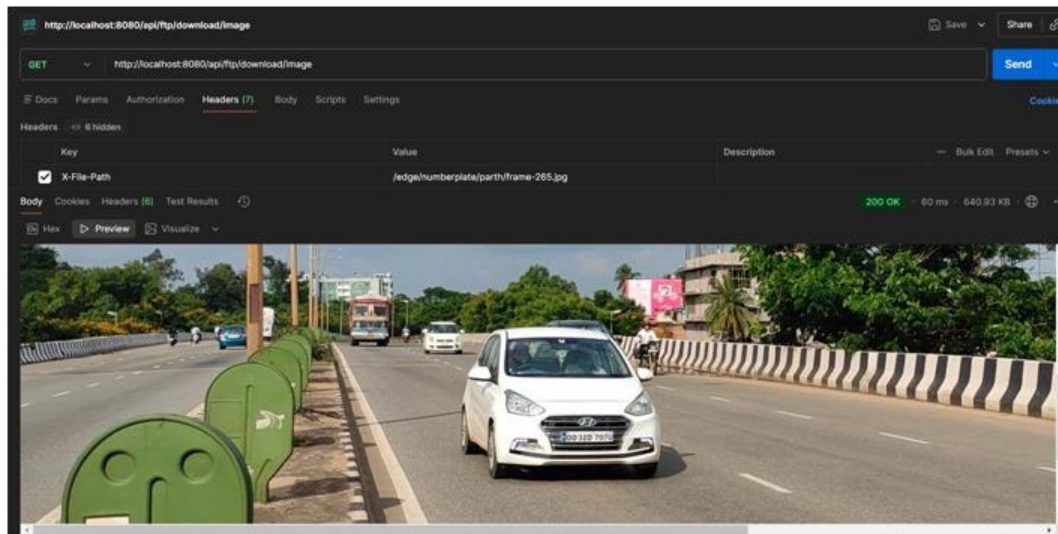
- **Dynamic File Uploading**

This **POST API endpoint** allows users to upload an image file along with a custom header (X-Ftp-Path) that specifies the destination path on the FTP server. The backend first verifies and creates any missing directories before storing the file. After the upload is completed, the image is processed by the number plate detection model, which generates an output image containing bounding boxes and detected labels.



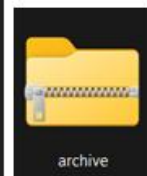
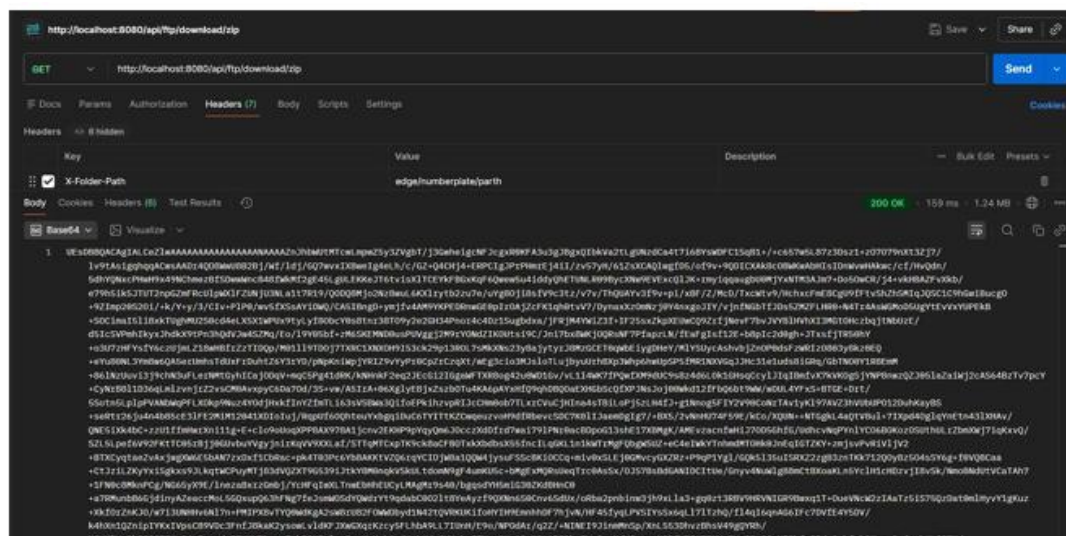
- **Direct Image Retrieval :**

This **GET API endpoint** is used to retrieve images directly from the FTP server. The client provides the file location through the X-File-Path header, and the backend streams the image data back to the client. The system automatically sets the correct HTTP **Content-Type** (such as image/jpeg or image/png) so that the image can be displayed properly in the browser or frontend application.



• On-the-Fly Folder Archiving :

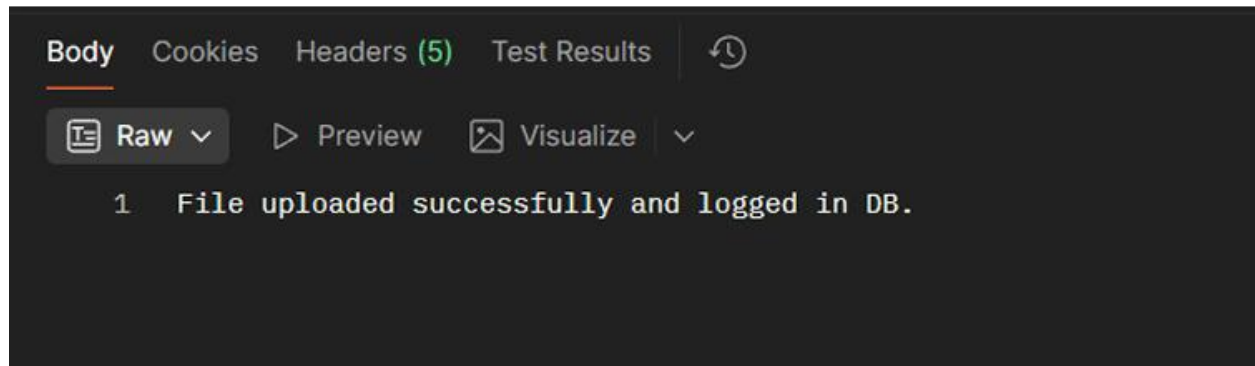
This endpoint enables users to download an entire directory from the FTP server as a **compressed ZIP archive**. The backend dynamically reads the files from the specified folder and streams them into a `ZipOutputStream`. The resulting archive is then sent to the client without storing any temporary files on the server, ensuring efficient and memory-optimized data transfer.



4. Database Auditing

To maintain proper tracking of file operations, a **database auditing system** was implemented. Whenever a file is uploaded successfully, the system creates a **FileAuditLog** record that stores details such as the file name, FTP storage path, and upload timestamp. This information is saved in the **PostgreSQL database**, providing a reliable history of all file uploads.

necessary **read and write permissions** to the Windows user account used by the backend application.



Conclusion

The developed backend system provides an efficient and scalable solution for managing file operations and number plate detection processing. By integrating RESTful APIs with an FTP-based storage system, the application enables smooth file uploads, retrieval, and organized storage management. The implementation of automated directory creation and database auditing further improves reliability by ensuring proper tracking of file activities.

Overall, the system demonstrates how modern backend technologies can effectively work with traditional FTP storage infrastructure, creating a secure and dependable platform for handling image processing and file management tasks.